HY559 TUNA: Reproducibility Project



Team members: Alexandros Fourtounis csdp1449

Background

Autotuning in modern systems:
- ML tuners assume stable performance. Works well in controlled environments.

Reality in the cloud:
- Virtualization, cache/memory interference. OS scheduling noise. Burstable VM credits.

Consequences:
- 5 % noise → 2.5× slower convergence.

TUNA Framework:

TUNA stabilizes the autotuning process by filtering and correcting noisy performance data before it reaches the optimizer.

Multi-Fidelity Sampling: Low-budget evaluation on single nodes → High-budget evaluation across distributed nodes.
Outlier Detection: Penalizes unstable configurations using Relative Range thresholds.
Noise Adjuster Model: A Random Forest regressor that "denoises" samples before they reach the optimizer

Objectives

Primary Goal

- Reproduce the key findings of the TUNA framework by deploying the distributed architecture on Microsoft Azure.

Core Pillars of Reproduction

1. **Infrastructure Replication:** Deploy the Orchestrator and Worker nodes on Azure using Ray and Dockerized containers.
2. **Operational Stability:** Stabilize the complex toolchain (Linux kernels, SSH auth, container networking) on modern Azure Ubuntu images.
3. **Experimental Validation:**
    - **Claim 1 (Noise):** Verify that artificial noise slows down convergence.
    - **Claim 2 (Efficiency):** Confirm TUNA finds optimal configurations faster than traditional Random Search (Baseline).

Target Metric:

- Generate a "Best Latency Found vs. Trials" trajectory curve for the TPC-C and Epinions benchmark.

Challenges

Authentication & Environment

- **Dependencies:** Missing GCC/PostgreSQL headers and Intel MLC license restrictions.
  - *Fix:* Manual installation of compilers and local transfer of patched MLC binaries.

Network & Container Isolation

- **"Ghost Code":** Ray workers served stale code cached in /opt/proxy rather than updated source.
  - *Fix:* Implemented strict synchronization to clear cache and restart Python servers.

- **DNS Resolution (Critical):** Workers failed to connect to DBMS containers due to Host Networking bypassing Docker DNS.
  - *Fix:* Redesigned topology to bind ports to 127.0.0.1 and rely on host loopback interfaces.

Implementation

Scaling Down for Constraints

- **Resource Limits:** Original paper used 32GB RAM nodes; we used 8GB RAM (Azure Standard_D2s_v3).
- **Workload Adaptation:**
  - Scaled down TPC-C factor from 100 (10GB) to 10 (≈1GB).
  - Reduced concurrent terminals from 40 to 5 to prevent starvation.
  - Implemented strict Docker memory caps (2.5GB Driver / 4.0GB Workload).

Codebase Modifications

- **Pandas Compatibility:** Patched AdjustedDistributedWorkerManager.py to fix TypeErrors where floating-point metrics were being saved into integer-initialized columns.
- **Configuration:** Created repro_pg16.json with relative paths and added mandatory metadata fields to prevent parser failure.
- **Data Analysis:** Developed custom Python scripts to consolidate multi-threaded CSV results and filter failed trials (score 999999).

Evaluation

Experimental Platform

- **Infrastructure:** 1 Orchestrator + 2 Workers (Standard_D2s_v3, 2 vCPU, 8GB RAM).
- **Software:** Microsoft Azure Platform. Minimal Ubuntu image.

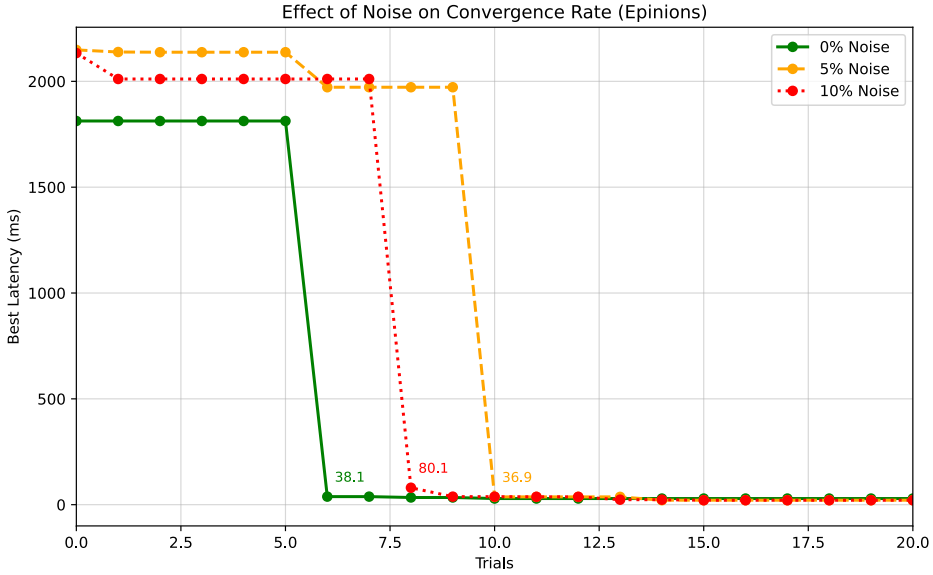
Experiment Phase 1: Noise Sensitivity

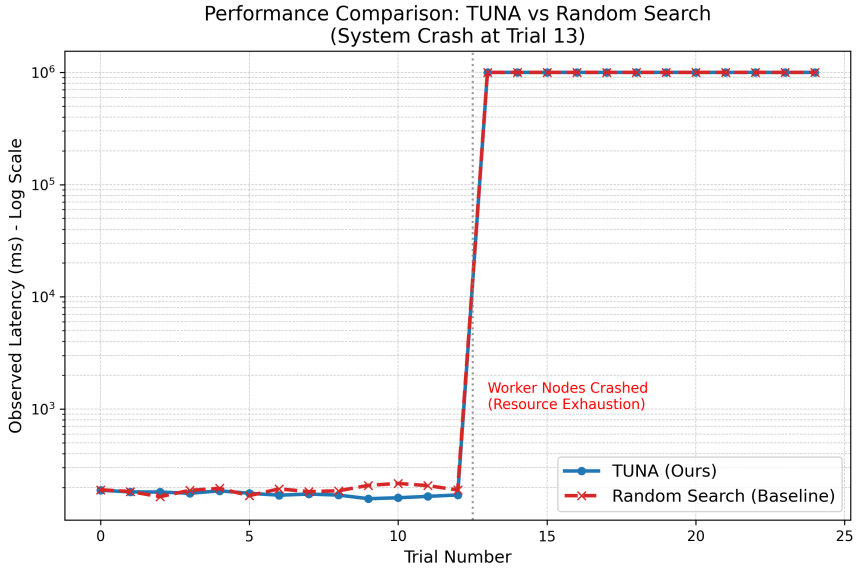
- **Goal:** Reproduce "noise equals slower convergence."
- **Setup:** Epinions workload via parallel_prior.py.
- **Variables:** Injected artificial noise at **0%**, **5%**, and **10%**.
- **Process:** 3 runs, each taking approximately 1 day to complete.

Experiment Phase 2: TUNA vs. Random Search

- **Goal:** Compare search efficiency.
- **Setup:** 25 Trials, 300s timeout.
- **Process:** Ran TUNA.py vs parallel.py (Baseline). Each took a significant time to complete.

Results





Noise Impact: Validated. 0% noise converged fastest. 5% and 10% noise trials delayed convergence, confirming noise degrades optimizer performance.

TUNA Efficiency: Validated. In the first 12 trials, TUNA reduced latency from **190ms** to **160ms** (**15.8%** gain). The Baseline failed to improve.

Crash happened due to resource exhaustion

Conclusions

- We successfully reproduced the claim that noise slows down the autotuner's convergence.

- We successfully reproduced the claim that TUNA navigates noisy environments better than baselines.

- However, we highlighted a critical need for **hardware headroom** as aggressive exploration can lead to catastrophic failure in resource-constrained clouds